

数字工程软件架构

第 4 章软件架构风格与模式笔记

基于 PPT/PDF 的图文整理笔记

原始课件：SA04_ 软件架构风格

整理者：InkTH

日期：2026 年 5 月 28 日

阅读说明

黑色正文表示来自课件的原始内容、同义压缩或忠实结构整理；浅蓝色提示框表示为帮助理解而补充的解释、例子或少量延伸。

目录

1	引言	2
2	软件架构风格定义	2
2.1	从模式到风格	3
2.2	软件架构风格的核心含义	3
2.3	定义与基本属性	3
2.4	风格与模式的区别	4
2.5	使用架构风格的优势	4
3	软件架构风格的分类	4
3.1	选择风格的依据	4
3.2	通用分类框架	5
3.3	分类时需要注意的问题	5
4	典型的软件架构风格	6
4.1	本章列出的典型风格全景	6
4.2	管道过滤器风格	7
4.3	主程序/子程序风格	8
4.4	面向对象风格	9
5	独立构件与虚拟机相关风格	11
5.1	事件驱动风格	11
5.2	解释器风格	12
5.3	仓库风格	14
6	网络化与复合架构风格	15
6.1	客户机/服务器风格	15
6.2	浏览器/服务器风格	17
6.3	异构风格	17

本节知识体系

本节知识体系

本章围绕**什么是软件架构风格、如何分类、哪些典型风格值得掌握**展开。

- 首先从引言切入，说明架构设计并不是每次都从零开始，而是可以从长期实践中抽象出可复用的组织模式。这些模式能够共享稳定结构，也能为团队交流提供统一术语。
- 随后给出**软件架构风格**的核心定义：它刻画的是一类系统共同拥有的构件、连接件、组合约束与语义特性，因此既是一套结构词汇，也是一组构建规则。
- 在分类部分，重点把握风格分析时需要回答的几个问题，以及 Garlan 和 Shaw 给出的通用风格分类框架。这里最重要的认识是：**风格是观察系统的视角，不是给软件贴唯一标签的标准**。
- 在典型风格部分，课程实际展开了九类代表风格：管道过滤器、主程序/子程序、面向对象、事件驱动、解释器、仓库、客户机/服务器、浏览器/服务器和异构风格。它们分别从**数据流、控制流、对象封装、事件传播、虚拟机解释、共享数据中心、网络分层协作以及多风格组合**等角度组织系统。
- 全章最后形成的判断标准是：选择架构风格必须围绕具体需求、限制条件、成本和风险，既看优点，也看代价；复杂系统经常不是单一风格，而是多种风格的组合。

1 引言

软件架构的一个重要特点，是可以从大量相似系统中**抽象出常见的、有共性的构建模式**。这些风格和模式来自长期开发实践中的总结、提炼与归纳，因此能够作为架构级重用的基础。

围绕这一点，本章强调了两个直接收益：

- **共享与复用**：能够复用架构风格中相对不变的部分，进一步支持软件平台、开源框架、低代码平台等更大粒度的复用。
- **沟通与理解**：只要说出系统采用 C/S、B/S、MVC、N 层、解释器或管道过滤器等常见术语，就可以在不展开实现细节的情况下，让相关人员迅速理解系统的大致组织方式。

一个设计良好的通用模式，往往也意味着某个工程领域的技术已经相当成熟。正因如此，风格与模式不仅是设计经验，更是工程知识被沉淀后的结构化表达。

2 软件架构风格定义

2.1 从模式到风格

从 Alexander 的表述出发，模式可以理解为**特定环境、问题与解决方案之间的关系**。它既是现实世界中反复出现的结构，又是指导这种结构如何被重复创造的规则。

因此，模式同时具有两面性：

- 一方面，它描述某种已经存在的组织形式，属于静态层面的“事物”。
- 另一方面，它告诉设计者在什么环境下、按照什么规则去构造这种组织形式，属于动态层面的“过程”。

2.2 软件架构风格的核心含义

一种软件架构风格刻画的是一个**系统家族**。这些系统共享某些结构和语义特性，因此能够用统一的术语和规则来描述。

具体地说，一种架构风格通常包含：

- **词汇表**：包括构件类型、连接件类型以及必要的元素。
- **配置规则**：规定这些元素允许如何组合，例如是否允许跨层调用、某类构件最多能连接多少个其他构件等。
- **语义解释**：说明各个成分的特性怎样共同决定系统整体特性。
- **分析依据**：使得基于该风格构建的系统能够被分析其优缺点、吞吐量、延迟、可维护性等属性。

换一种更直观的说法，软件架构风格就是：

- 一组构件的集合；
- 一组连接件及交互方式的集合；
- 一套组织系统的共性规则。

2.3 定义与基本属性

软件架构风格又可称为**架构惯用泛型**，它描述某一特定应用领域中系统组织方式的惯用模式，作为可复用的组织模式和习语，为设计交流提供公共术语空间，并促进设计复用与代码复用。

其基本属性可以概括为四项：

1. 设计元素的词汇表；
2. 元素组合的配置规则；
3. 元素组合的语义解释；
4. 基于该风格构建系统时的相关分析。

2.4 风格与模式的区别

Compare

架构风格更强调系统整体结构框架，关注的是系统在组织层面“长成什么样”；**架构模式**更面向具体问题，强调某类需求问题的结构性解决办法。

本章给出的区分重点有三条：

- 架构风格主要描述应用系统的总体结构框架。
- 架构风格相对独立于特定问题，更偏向通用组织形式。
- 架构模式比架构风格更加面向问题，强调问题一解决方案对，并隐含“追求最佳效果”的倾向。

Explanation

可以把风格理解为“组织系统的一般说法”，把模式理解为“针对某类问题时更有针对性的结构答案”。两者经常重合，但关注重点并不完全相同。

2.5 使用架构风格的优势

使用标准化架构风格的主要优势包括：

- 极大促进设计重用与代码重用；
- 使系统组织结构更容易理解，便于人员沟通；
- 即使没有实现细节，也能大致推断系统组成和工作方式；
- 便于支持特定分析，例如对管道过滤器风格分析调度、吞吐量、延迟和死锁等问题；
- 更好支持系统内部互操作性。

3 软件架构风格的分类

3.1 选择风格的依据

架构风格为大粒度软件重用提供了可能，但具体项目并不存在“永远最好的风格”。风格的选择必须围绕**需求、限制条件、复杂度、难度、成本与风险**进行分析比较。

分析某种风格时，需要回答以下问题：

- 构件和连接件的类型是什么；
- 可容许的结构模式是什么；
- 基本计算模型是什么，如单机、网络、Web、云、移动等；
- 风格的基本不变性是什么；
- 常见实例、优缺点与特例有哪些。

这些问题最终仍回到四个核心要素：**词汇表、配置规则、语义解释原则、风格分析方法**。

3.2 通用分类框架

Garlan 和 Shaw 给出的通用软件架构风格分类包括五大类：

- **数据流风格**：批处理序列、管道过滤器；
- **调用和返回风格**：主子程序、面向对象系统、分层结构；
- **独立构件风格**：通信进程、事件系统；
- **虚拟机风格**：解释器、基于规则系统；
- **仓库风格**：数据库、超文本系统、黑板。

通用软件架构风格分类

数据流风格	调用和返回风格	独立构件风格	虚拟机风格	仓库风格
批处理序列	主子程序	通信进程	解释器	数据库
管道过滤器	面向对象系统	事件系统	基于规则系统	超文本系统
	分层结构			黑板

图 1 通用软件架构风格的分类框架

随着研究与应用的发展，风格仍在不断扩展，例如 C2 风格、GenVoca 风格、REST 风格，以及面向分布式系统、交互式系统、适应性系统等提出的更多变体。

3.3 分类时需要注意的问题

不同风格是从**不同视角或不同抽象层次**总结出来的，因此它们之间常常存在交叉现象。

- 主程序/子程序和面向对象风格，是大量软件构件与连接件设计实现的基础。
- 层次化风格，几乎是一切复杂系统的基本结构。
- 事件驱动风格，体现的是一种由系统管理控制的构件连接方式。
- 仓库风格，体现的是以数据库或知识库为中心的组织结构。
- 管道过滤器、客户机/服务器、解释器等，更像是某些特殊场景下的典型组织方式。

因此，架构风格**不是对软件进行唯一分类的标准**，而是描述软件的不同角度。同一个系统完全可能同时采用多种风格，形成异构组合。

Tips

这也是为什么一个系统可以同时说自己是“分层的”、又是“面向对象的”、还采用了“事件驱动”的。它们说的并不是同一层面的结构事实。

4 典型的软件架构风格

4.1 本章列出的典型风格全景

本章列出的典型风格共有二十类：

1. 管道过滤器风格
2. 主程序/子程序风格
3. 面向对象风格
4. 层次化风格
5. 事件驱动风格
6. 解释器风格
7. 基于规则的系统风格
8. 仓库风格
9. 黑板系统风格
10. C2 风格
11. 客户机/服务器风格
12. 浏览器/服务器风格
13. 平台/插件风格
14. 面向 Agent 风格
15. 面向方面软件架构风格
16. 面向服务架构风格
17. 正交架构风格
18. 异构风格
19. 基于层次消息总线的架构风格
20. 模型-视图-控制器风格

其中，课件实际展开分析的重点集中在下列九类风格。

4.2 管道过滤器风格

4.2.1 基本思想与结构

在管道过滤器风格中，每个功能模块都有一组输入和输出。

- **构件**是过滤器，即各个独立完成处理的功能模块。
- **连接件**是管道，即输入输出数据流之间的通路。

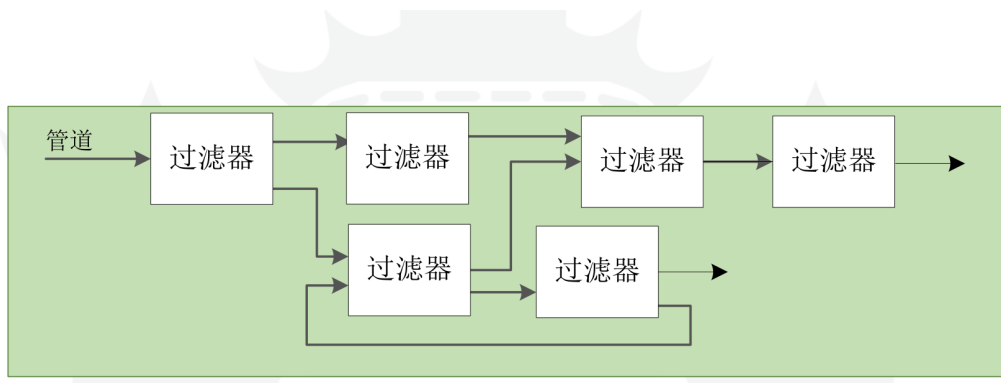


图 2 管道过滤器风格中的过滤器与管道组织方式

这种风格的关键特性包括：

- 过滤器相对独立，独立完成自身功能；
- 非邻近过滤器之间不共享状态；
- 过滤器自身通常无状态；
- 各过滤器对其上下游过滤器“无知”；
- 结果正确性不依赖某个中心控制者，而依赖过滤器拓扑结构与数据流组织。

4.2.2 优点与缺点

优点主要有：

1. 系统行为容易理解；
2. 支持功能模块重用；
3. 易于维护和扩展，新过滤器容易加入，旧过滤器容易替换；
4. 便于进行吞吐量、死锁等专门分析；
5. 支持并发执行。

缺点主要有：

1. 处理过程往往偏成批顺序执行；
2. 不适用于强交互式应用；
3. 在数据传输中可能被迫采用底层公共命名，增加解析与反解析负担。

4.2.3 实例

典型实例包括 Unix/DOS 管道命令、传统编译器处理链，以及遥感影像处理的一般流程。

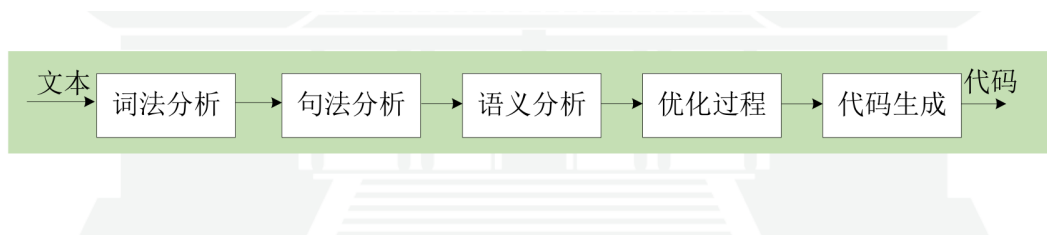


图 3 传统编译器视为典型的管道过滤器处理链

Example

把词法分析、句法分析、语义分析、优化和代码生成串联起来时，每个阶段都像一个过滤器，前一阶段输出直接成为后一阶段输入，这正是数据流风格最典型的表现。

4.3 主程序/子程序风格

4.3.1 基本思想

主程序/子程序风格产生于结构化程序设计盛行之后。它从**功能观点**设计系统，通过逐步分解和逐步细化得到系统架构。

- 构件是主程序和子程序；
- 连接件是调用-返回机制；
- 拓扑结构是层次化结构。

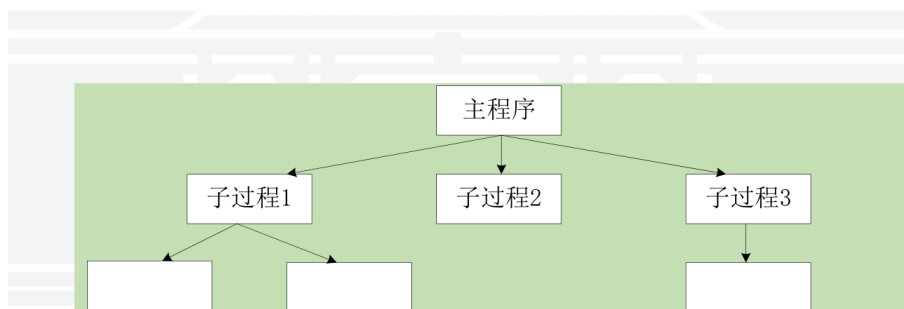


图 4 主程序/子程序风格中的层次化调用结构

4.3.2 优点与缺点

优点包括：

1. 数据访问效率高，因为各计算往往共享同一个存储区；
2. 不同计算功能被划分到不同模块，结构比散乱代码更清晰。

缺点集中在**变更适应性差**：

1. 数据存储格式变化会影响大量模块；
2. 处理流程变化和功能增强难以适配；
3. 对复用支持较弱，过度依赖既有数据结构与调用顺序。

4.3.3 KWIC 实例

KWIC (Key Word in Context) 系统要求对输入语句进行循环移位并排序输出，是该风格的经典示例。

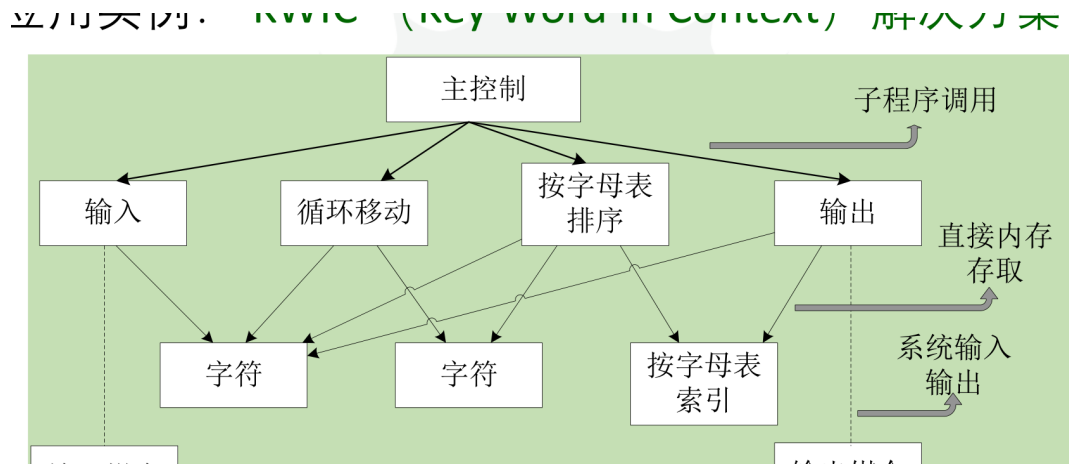


图 5 KWIC 系统在主程序/子程序风格下的组织方式

Compare

与管道过滤器相比，这里强调的是**中心控制和调用顺序**；前者强调的是**数据流经各阶段时的独立处理**。

4.4 面向对象风格

4.4.1 基本思想

面向对象风格从现实世界中的事物出发，把问题域中的实体抽象为对象，并将对象作为系统的基本构成单位。

其两个重要特点是：

1. 对象负责维护自身表示的完整性，即状态与行为；
2. 对象的表示对其他对象是隐藏的，即封装性。

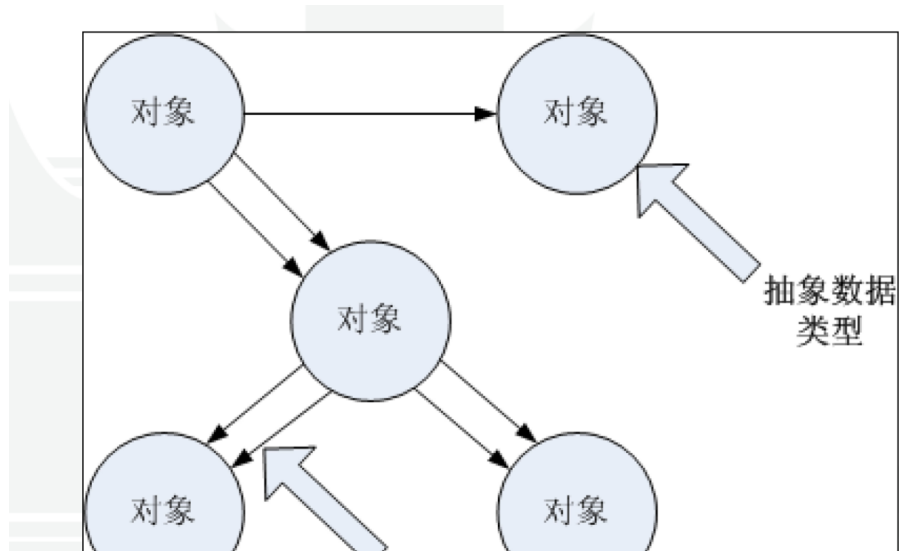


图 6 面向对象风格下以对象交互构成系统结构

4.4.2 适用场景、优点与缺点

这种风格适用于**数据和功能捆绑明显**的系统，也适用于问题域模型较明显或需要人机交互界面的系统。很多事件驱动系统也会同时采用面向对象组织方式。

优点主要有：

1. 对象隐藏实现细节，修改对象实现时对外部影响较小；
2. 使用方式更简单，且有利于安全性、可靠性；
3. 便于把数据访问和行为组织为交互代理集合。

缺点是：

- 当对象通过过程调用等方式交互时，必须知道其他对象标识；
- 一旦对象标识改变，所有显式调用它的对象都可能需要修改。

4.4.3 KWIC 实例

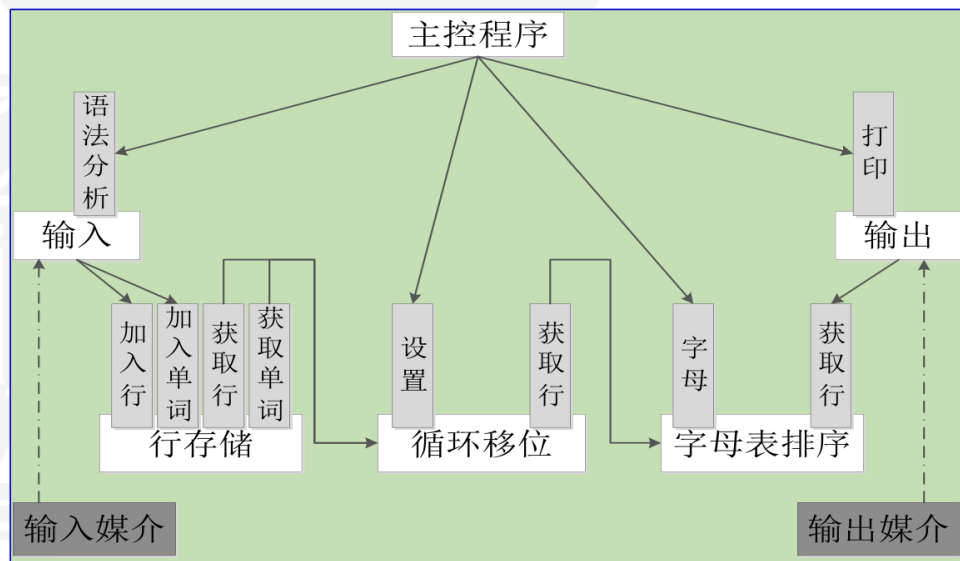


图 7 KWIC 系统在面向对象风格下的对象划分与调用关系

5 独立构件与虚拟机相关风格

5.1 事件驱动风格

5.1.1 基本思想

事件驱动系统中的构件通常包含两类内容：

- 一组过程集合，可以被显式调用，也可以在事件中注册后被隐式触发；
- 一组事件集合。

这种风格中，构件并不直接调用某个处理过程，而是**发布、广播或声明事件**。其他构件通过注册表达对事件的兴趣，当事件发生时，系统自动调用已注册的处理过程。

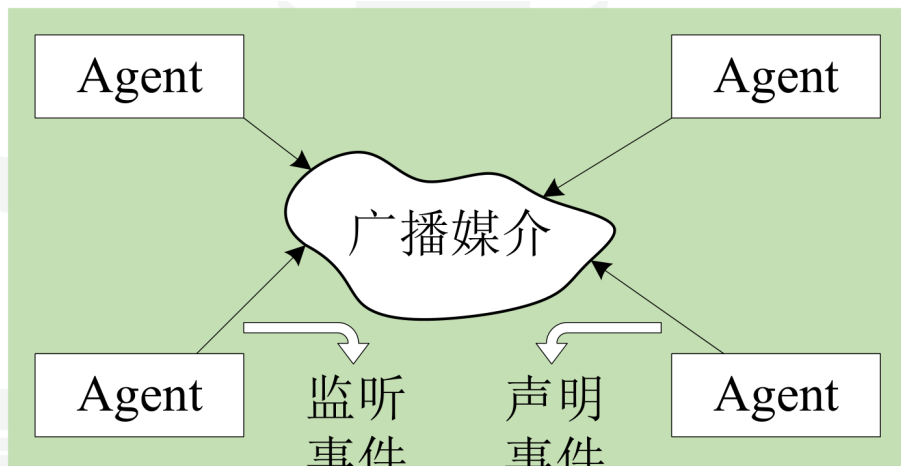


图 8 事件驱动风格中的广播媒介、声明事件与监听事件关系

5.1.2 优点与缺点

优点包括：

1. 事件声明者不需要知道会影响哪些构件，耦合较弱；
2. 事件发布与处理逻辑都具备较强复用性；
3. 易于增加新事件和新处理过程，便于升级。

缺点包括：

1. 构件放弃了部分控制权，由系统决定触发时机；
2. 数据交换与共享变得更难组织；
3. 系统对象间逻辑关系更复杂；
4. 正确性验证更困难，尤其是事件处理顺序与效率问题。

5.1.3 实例

典型实例包括 FIELD 系统与 Windows GUI 程序。

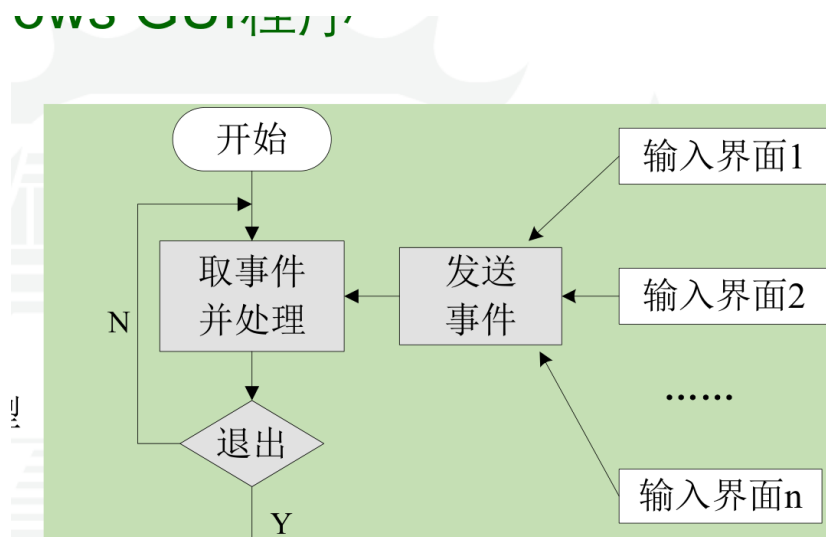


图 9 Windows GUI 程序中的典型事件循环模型

Explanation

事件驱动的关键不是“有消息”，而是事件发生后由系统负责调度相应处理者。这使得界面、编辑器、网络通知等场景都非常适合采用这种组织方式。

5.2 解释器风格

5.2.1 基本思想

解释器是一个用来执行其他程序的程序。它在硬件平台之上实现一个虚拟机，把高抽象层次程序翻译为低抽象层次平台所能理解的指令，从而弥合程序语义期望与硬件计算引擎之间的差异。

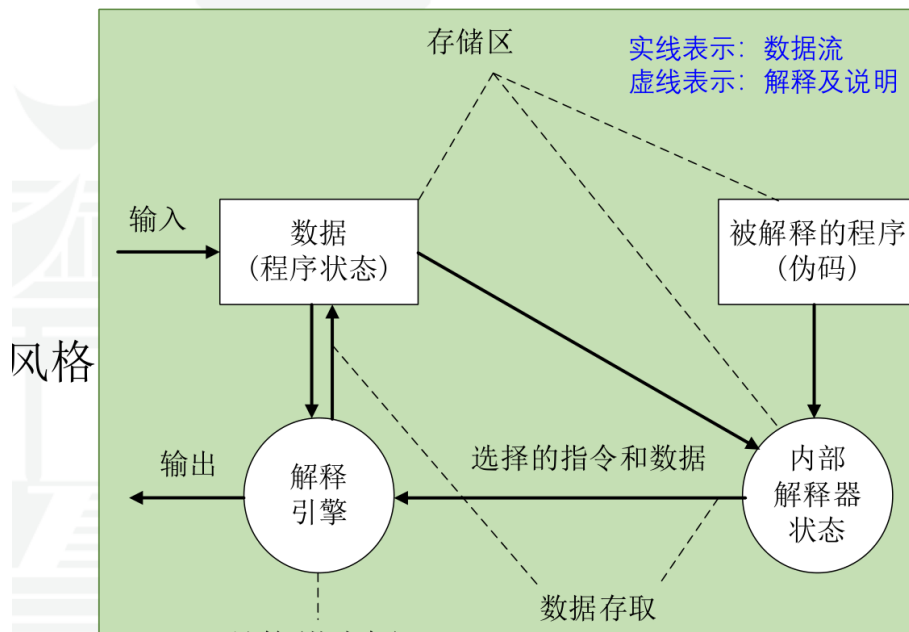


图 10 解释器风格的组成部分与数据流

其核心组成通常包括：

- 被解释程序；
- 程序状态数据；
- 内部解释器状态；
- 解释引擎；
- 存储区与输入输出。

5.2.2 优点与缺点

优点包括：

1. 有利于程序可移植性和语言跨平台能力；
2. 有利于对未来硬件进行模拟和仿真，降低测试复杂性和成本。

缺点是：

- 额外的间接层次会带来性能下降；
- 如果不引入 JIT 等机制，运行速度通常低于直接编译执行方式。

5.2.3 JVM 与 JIT 实例

JVM 是典型解释器风格实例。Java 源码先编译成字节码，再由 Java 平台解释执行；如果引入 JIT，则可以把字节码即时编译为目标码后再执行。

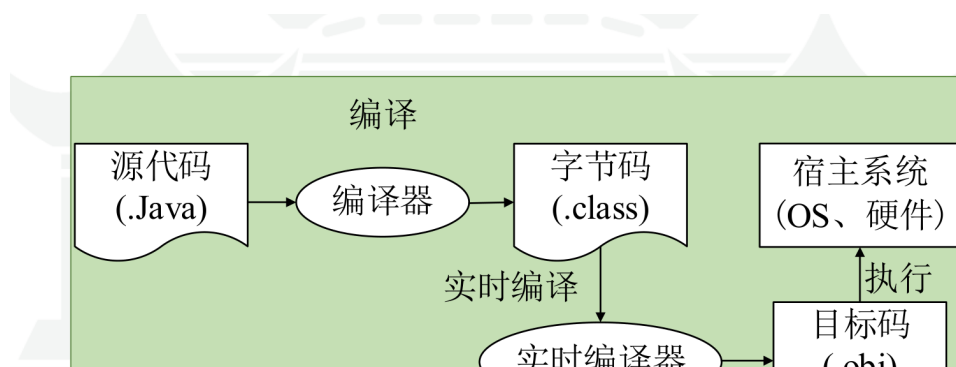


图 11 JIT 方式下字节码被实时编译后再交由宿主系统执行

Tips

解释执行强调跨平台与灵活性；JIT 的加入，是在保持虚拟机体系基础上，尽量把一部分性能损失拿回来。

5.3 仓库风格

5.3.1 基本思想

仓库是存储和维护数据的中心场所。仓库式风格由两种构件组成：

- 中央数据结构构件；
- 相对独立的构件集合。

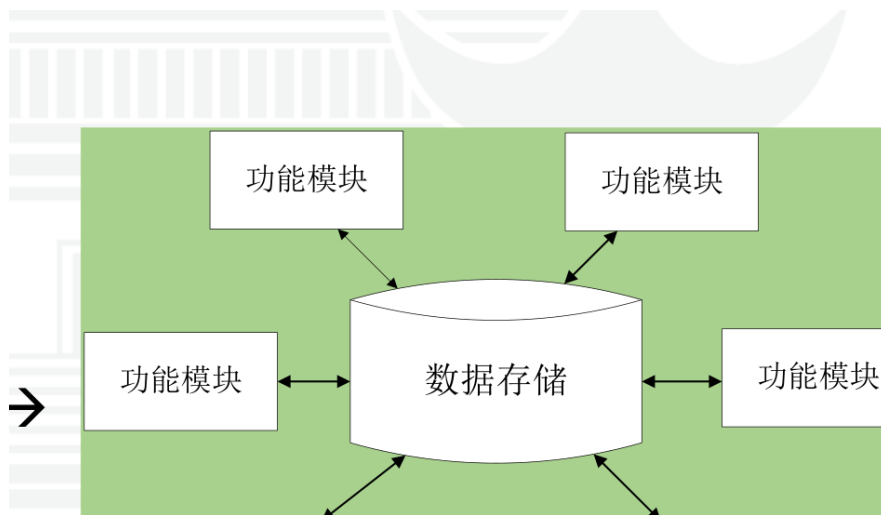


图 12 仓库风格中以中央数据存储为核心的组织方式

5.3.2 优点与缺点

优点包括：

1. 便于模块间共享数据；
2. 方便模块添加、更新和删除；

3. 避免知识源或数据被不必要地重复存储。

缺点在于：

- 必须借助同步、加锁等机制维护数据完整性与一致性；
- 中央数据结构往往成为关键资源，也可能成为系统瓶颈。

5.3.3 实例

典型实例包括现代编译器和基于仓库风格的软件研发环境 Eclipse。

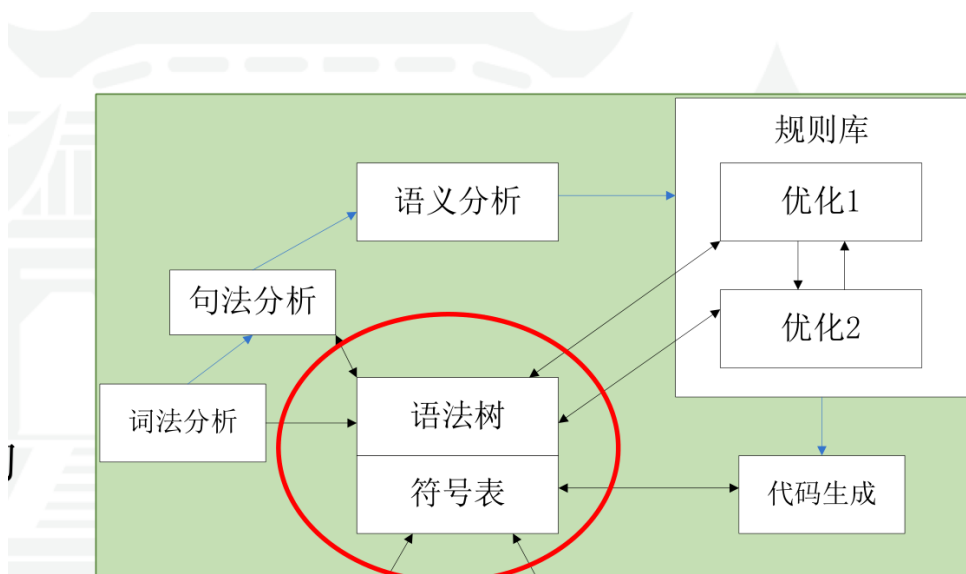


图 13 现代编译器中围绕语法树、符号表和规则库组织功能模块

6 网络化与复合架构风格

6.1 客户机/服务器风格

6.1.1 基本思想

客户机/服务器（Client/Server）风格主要针对资源不对等问题提出，是一种共享策略。客户机与服务器是两个相互独立的逻辑系统，通过请求/响应完成协作。

- 客户机位于前端，负责业务逻辑的一部分以及与服务器通信的接口；
- 服务器位于后端，负责与客户机通信、业务处理和数据管理。

客户机与服务器可以通过消息、共享存储区、信号量、RPC 以及 HTTP、POP、SMTP、FTP 等协议通信。

6.1.2 两层与三层 C/S

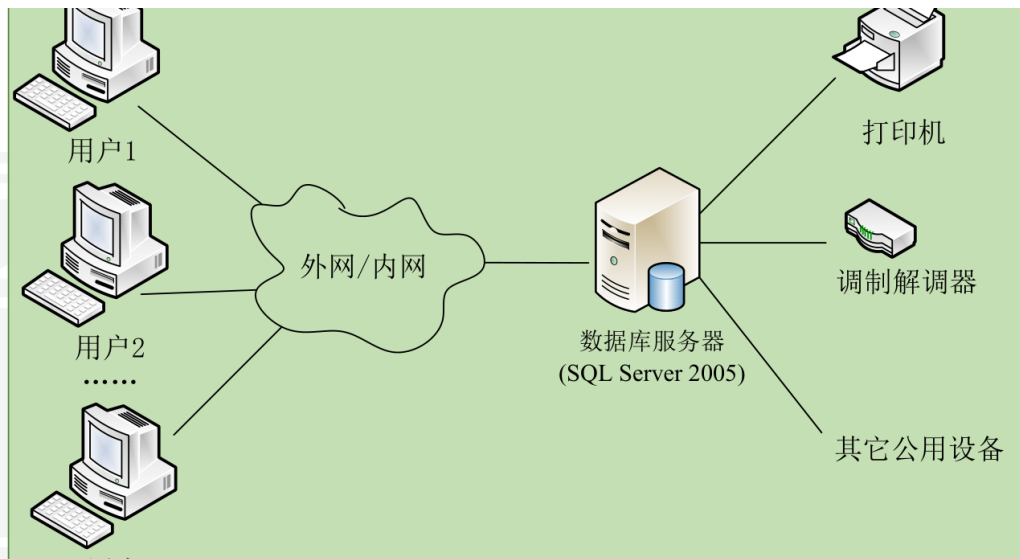


图 14 两层 C/S 架构中客户机直接通过网络访问数据库服务器

两层 C/S 的优点包括：

1. 有利于分布式数据组织与处理；
2. 构件位置相互透明；
3. 便于在异构操作系统与多种技术之间融合；
4. 环境配置灵活，易于扩展；
5. 能把业务逻辑分散到多台低成本计算机上。

其缺点包括客户端负担重、升级维护成本高、扩展到 Internet 困难以及数据安全性较弱等。

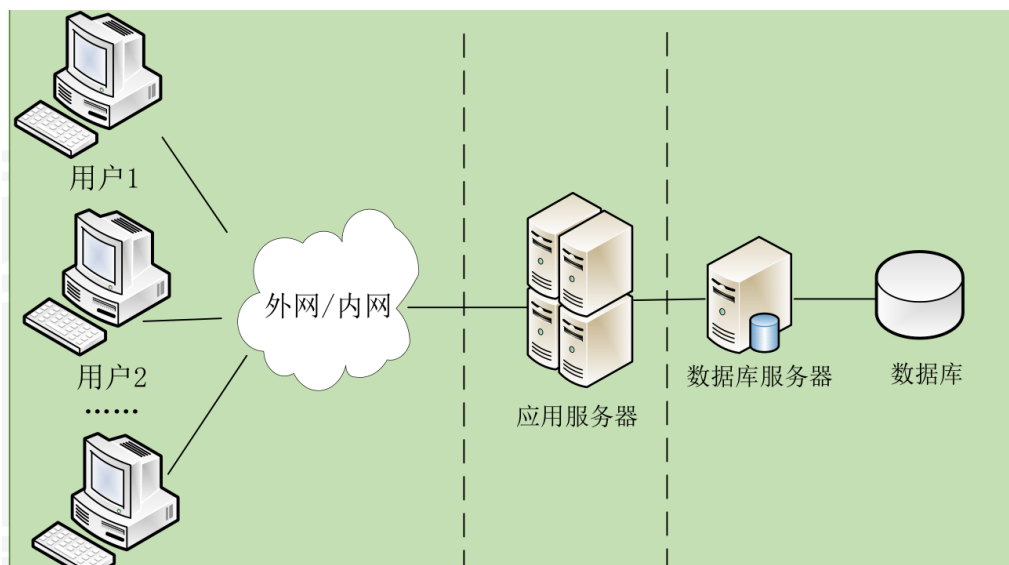


图 15 三层 C/S 架构把应用服务器单独抽出形成中间功能层

三层 C/S 相比两层 C/S 的改进主要体现在：

- 功能划分更清晰；
- 可维护性和可扩充性更好；
- 更便于为不同层选择合适平台；
- 并行开发与安全性更好。

但实现三层 C/S 时必须认真处理层间通信方式、通信频率和传输数据量，否则整体性能会受限。

6.2 浏览器/服务器风格

浏览器/服务器 (B/S) 风格是三层 C/S 的一种实现方式，主要由浏览器、Web 服务器、数据库服务器构成。

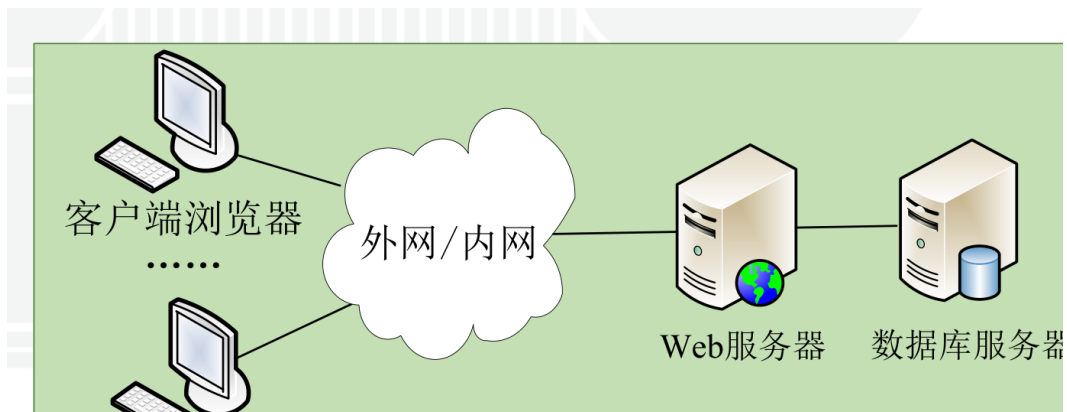


图 16 B/S 架构以浏览器作为客户端、Web 服务器作为应用服务层

其主要特点是：

- 客户端只需要浏览器；
- Web 服务器承担数据请求、网页生成、数据库访问和应用程序执行；
- 安装、修改和维护主要集中在服务器端完成。

优点包括跨平台通信、客户端操作简单、维护集中、开发成本相对较低。缺点包括个性化程度较低、客户端数据处理能力弱、动态交互受限、扩展性与安全性存在压力，以及访问中心数据库的速度往往低于 C/S。

Note

B/S 和 C/S 的差别，不只是“一个用浏览器、一个不用浏览器”。更本质的区别在于：客户端职责是否被极度收缩，并把更多应用逻辑统一收敛到 Web 服务器一侧。

6.3 异构风格

6.3.1 基本思想

异构架构是几种风格的组合。由于不同风格适用于不同问题，在真实系统中，多种架构共存和相互融合往往是常态。

常见组合方式有两种：

1. 系统总体采用层次结构，但每一层内部又采用不同风格；
2. 单一构件中允许复合连接件，例如把事件驱动与直接调用结合使用。

6.3.2 优点与缺点

优点是：

- 有利于遗留代码重用；
- 能在共享标准与软件包的前提下，兼容不同解释习惯和表示方式；
- 更适合复杂现实系统中“内外有别、查改有别、设备条件不同”的组织需求。

缺点是：

- 不同风格之间的兼容问题有时很难解决；
- 系统整体理解难度往往高于单一风格架构。

6.3.3 实例

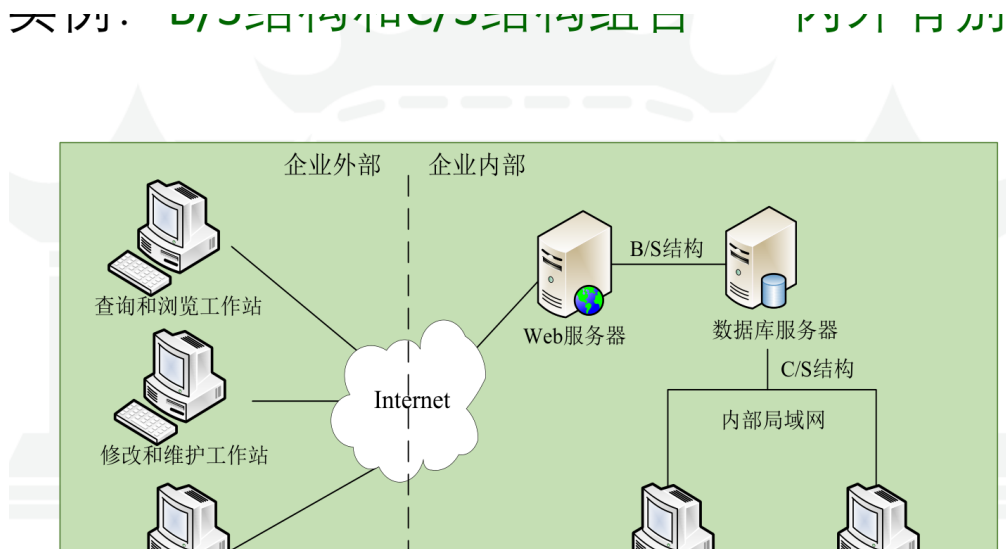


图 17 B/S 与 C/S 组合形成“内外有别”的异构架构

典型实例包括：

- B/S 与 C/S 组合形成“内外有别”模型；
- B/S 与 C/S 组合形成“查改有别”模型；
- 供电管理系统中把外部访问和内部控制组织为不同风格的组合框架。

Explanation

异构架构并不意味着“架构不统一”，而是承认复杂系统不同区域面对的问题不同，因此允许在统一大框架下局部采用更合适的风格。